

Object Oriented Software Engineering

Dagmawi Lemma (PhD)

Assistant Professor, AAU

courses.dag@gmail.com

<https://sites.google.com/site/dagmawi2012/>

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

1

1. Introduction

Software Engineering

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

2

From Programming to Engineering

- Originally computers were considered just as a machine that is meant to be **configured** to solve a particular problem.
- Later the required configuration revealed itself as programming.
- As computers become a general purpose machine, the importance of software becomes indispensable
 - Also the development process began to involve several stakeholders who have diversified, and even conflicting, requirements

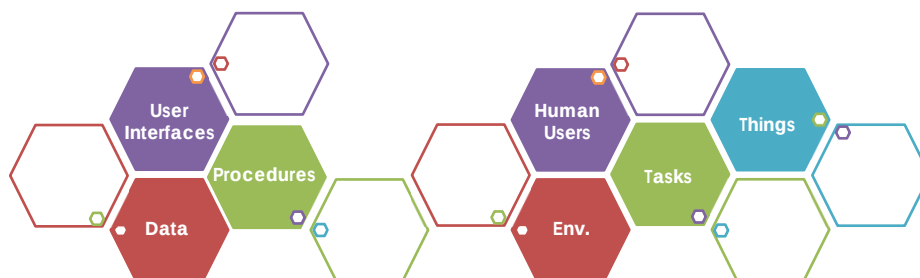
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

3

Software Elements

- Software in general refers to the program that runs in the computer.
- But nowadays, it is more than that, and each of the element will have implication on how we think about software and how we develop it, as well as while considering the platform/environment on/in which it runs.



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

4

From Programming to Eng'...

- Furthermore, as software become core part of computer and many business, organizations, institutes, etc. are relying on computers, thus faults in software are no longer tolerable
 - Often the fault was rooted to developing software with erroneous requirements:
 - Missing user requirements
 - Wrong understanding (assumption) of user requirements
 - Implementing undesired requirements (feature)
- The situation leads to what is known as **software crisis** and amplified the need for engineering approach.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

5

Software Crisis

- The time of intense and the looming danger and challenges that resulted from lack of effective software development leads to crisis and the situation was considered as **software crisis**
- Software crisis was a term coined in used to express the difficulty of developing **useful, effective, and efficient** software within specific time and budget; and manifested in several ways:
 - Projects running over-budget
 - Projects running over-time
 - Software was very inefficient
 - Software was of low quality
 - Software often did not meet requirements
 - Projects were unmanageable and code difficult to maintain
 - Software was never delivered
- As a result of the challenges faced in developing software, which is efficient and effectively solve the user's problem, it was noted that software development need to be done as a process that follows engineering approach – hence software engineering evolve as a discipline.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

6

What is Software Engineering?

- To begin with, what is engineering?
 - It is about problem solving
 - Note that you solve problem in various ways
 - E.g., discussion, supply the need, etc.
 - So how is engineering solve a problem?
 - Examples
 - » A problem of accessing city leads to constructing road
 - » A problem of shelter leads to building houses
 - » A problem lifting heavy objects to a height leads to construct crane,
 - » A problem of organizing data leads to database system, etc.
 - **Engineering solves a problem by delivering a product.**

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

7

What is SE? (*continued*)

- Engineering is a **process** of developing a **product** by applying mathematical and scientific methods with the intention of **solving a well defined problem**.
- Software Engineering is a **process** of developing a **software** by applying mathematical and scientific methods with the intention of solving **problem** by improving efficiency in the use of computer while performing some **predefined user task**.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

8

The Problem

- Problem to be solved through engineering is often related to:
 - Inability to do some task due to lack of capacity, and limitation of human beings in performing their task or some activity.
 - E.g., calculating complex mathematical equation, remembering booking detail, etc.
 - Aspiration to efficient and better way of accomplishing task
 - E.g., performing routine job diligently, supporting large number of customers cost effectively, etc.
- Hence the problem to be solved through **engineering aims to improve human capability and efficiency while users (individuals or organizations) perform various activity.**

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

9

The Problem (*continued*)

- Most software development projects are initiated:
 - with the intention of solving users' problem to support users perform their task/s
 - which are not humanly possible
 - E.g., it is humanly not possible to remember all the expiry dates of drugs shelved in a drugstore
 - efficiently and effectively by increasing productivity, safety, satisfaction, etc.

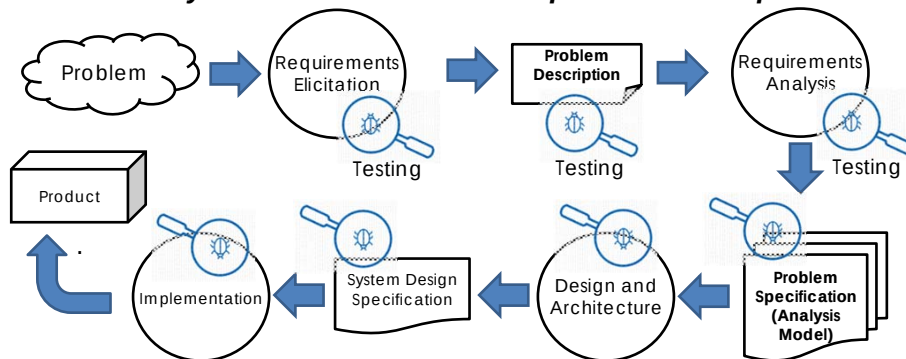
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

10

The Process

- In engineering, a process is a series of interrelated tasks, which are needed to develop/construct a certain **product**.
- These series of tasks, together, transform inputs into a given output leading to the final output (i.e., the product) or the tasks **may meant to maintain and fix problems in the product**.



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

11

Software Development Approaches

- Two possible views we can have about software;
 - **The data:** that is expected to be used and/or maintained by the software.
 - **The function or procedure:** that process the data as per some business logic and changes the state of the system.
- Accordingly, the software development approach might follow two possible approaches that considers the data and the function in different perspective.
- A system is a collection of various elements those abide to some rule and works/collaborate together to achieve some goal.
 - Software, being a system, what are the elements in it?

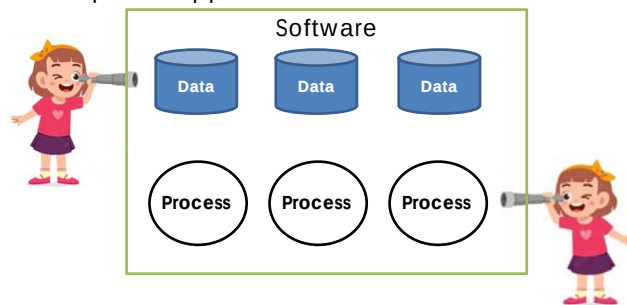
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

12

Software Development Approaches

- Perspective 1: software is a collection of functions (procedures) those work together to achieve some common goal.
 - Thus the data and the function shall be studied and designed separately.
 - Having such consideration, we can follow procedural (structured) software development approach.



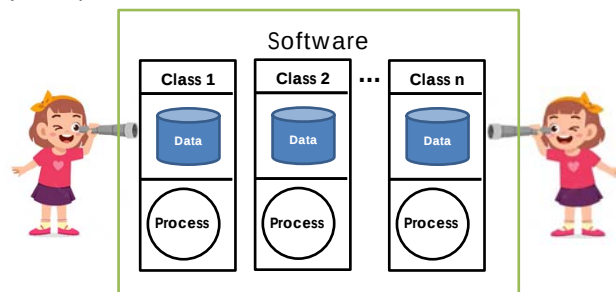
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

13

Software Development Approaches

- Perspective 2: software is a collection of objects where the object contains a function that manipulates data of the respective object; these objects interact to one another achieve some goal.
 - Thus the data and the function that are contained in each object shall be analyzed and designed by identifying the objects which contain those data and function.
 - Having such consideration, we can follow object oriented software development approach.
 - Technically, an object is instance of a class



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

14

Software Development Approaches

- Therefore, broadly we classify the software development approach into two:
 - Structured/Procedural
 - The process, the role and the data are separately studied (analyzed, designed, implemented, and tested)
 - Objects
 - Assumes that software is collection of objects interacting one another to solve the problem.
 - The objects collaborate
 - Objects are instance of class that comprises of the process and data, while roles are considered as external players known as actors.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

15

The Water Fall

“Way back when”

- Royce, W. W. (1970, August). Managing the development of large software systems. In *proceedings of IEEE WESCON* (Vol. 26, No. 8).
 - essential steps common to all computer program development
 - Analysis and Coding
 - System requirement, software requirement, analysis, design, coding, testing, operation
 - Iterative relationship between successive phases

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

16

Types of Process Model

- Three generic process models (Sommerville **2007**)
 - The waterfall model
 - Evolutionary development
 - Component-based SE
- Or a process model can be linear, incremental, agile

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

17

Types of Process Model

- In principle, the result of each phase in any of the process model is one or more documents that are approved.
 - In practice there is no clear cut rather the phases overlap
 - The cost of producing and approving documents as well as the number of iterations are expensive
 - Freezing parts of the development ☹
 - Problems are left for later resolutions
 - Ignore the problem
 - Solve the problem by programming around (without documentation)

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

18

Types of Process Model

- Waterfall model
 - Logically ordered as in SDLC
 - It is however practical if the requirements are well understood and unlikely to change radically during the process
- But what about the documentation?
 - Documentation is often rigorous and bulky
- Other approaches also seems to practice the waterfall but by considering small portion of the entire system
 - Hah! The water fall seems to appear as sub model
 - No! the SDLC is reflected rather

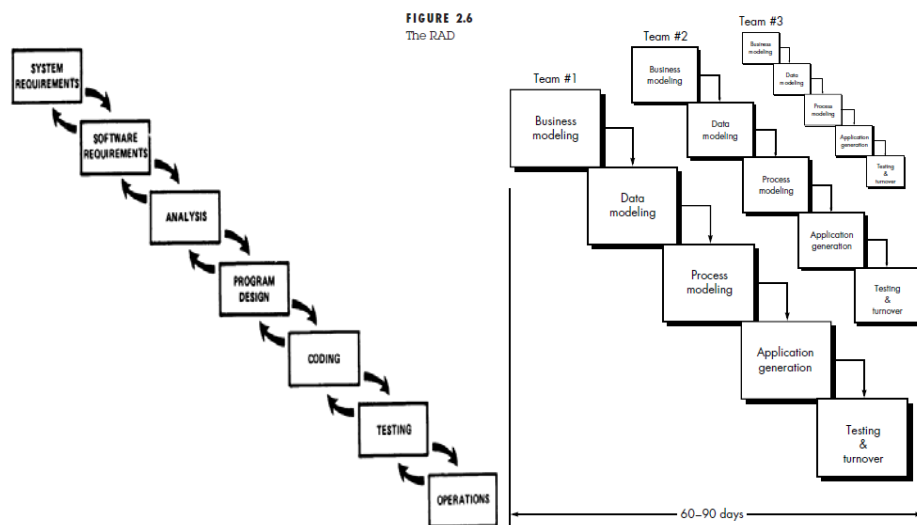
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

19

Types of Process Model

FIGURE 2.6
The RAD

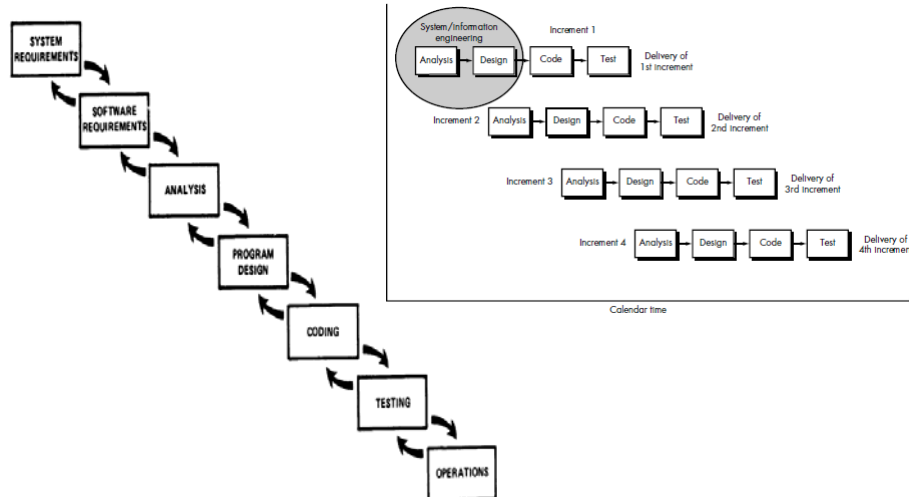


Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

20

Types of Process Model



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

21

Types of Process Model

- If we still have the sequence of the activities in RAD, Incremental, Spiral, etc. as in the waterfall, why do we consider them as different process model?

– REMARK:

- Process descriptions is more than the sequence of activities: it may also include principles to be followed and description about:
 - Products
 - Roles
 - Pre- and post-conditions,
 - tools and techniques etc.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

22

The Paradigm shift

- Some of the reason why software project fails
 - Poorly documented client requirements
 - When do we say it is poorly documented?
 - Impractical schedule
 - Change management problems
 - Testing issues
 - Overly rigid processes

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

23

Agile methods

- The overhead cost of software development could be extensive
 - The cost can be in planning, documenting, managing, etc.
 - For small and medium-sized business systems, the overhead involved could be large enough and sometimes dominates the process.
 - More time could be spent working on how the system should be developed rather than on the program development and testing
 - The overhead can be easily justified for heavyweight systems but not for small and medium-sized systems

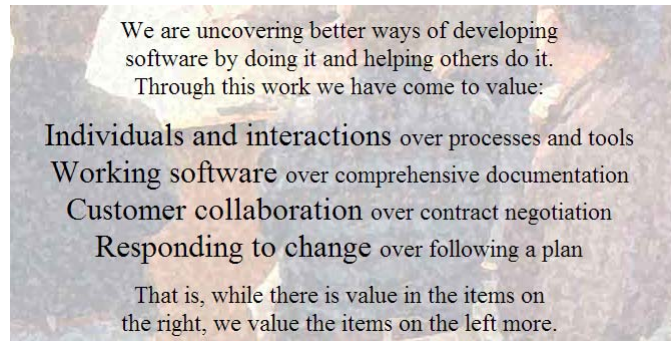
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

24

Agile methods

- The agile manifesto
<http://agilemanifesto.org/iso/en/>



Dagmawi L.
 (courses.dag@gmail.com)

Software Engineering

25

Agile methods

- Agile manifesto
 - Collaboration
 - Communication
 - Focus on people instead of on the process
 - Delivering a working software is a measure of success
 - We need the outcome not how we did the outcome

Dagmawi L.
 (courses.dag@gmail.com)

Software Engineering

26

Agile methods

- Long milestones → short ones (sprint)
- Extensive documentation → less documentation
- Process overhead → product delivery
- Evolved as SD Process and management tool

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

27

Agile methods

- **Principles behind the Agile Manifesto**
 1. satisfy the customer through early and continuous delivery of valuable software
 2. At anytime in the development process welcome changing requirements
 3. Deliver working software frequently, from a couple of weeks to a couple of months
 4. Work together daily throughout the project.
 5. Motivate team members
 6. Face-to-face conversation
 7. Measure of progress by the working software
 8. Maintain a constant pace between stakeholders
 9. Continuous attention to technical excellence and good design enhances agility.
 10. Simplicity--the art of maximizing the amount of work not done--is essential.
 11. The best architectures, requirements, and designs emerge from self-organizing teams.
 12. Reflects on how to become more effective

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

28

Agile methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritise new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognised and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change, so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

29

Agile methods

- Best practice
 - Design a clear, concise user story
 - Employ business value prioritization
 - Reflect on the project
- To Success
 - Define Agile principles best suited for the project (organization)
 - Establish and define a glossary of terms
 - Over communicate

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

30

Agile methods

- The case of **SCRUM**
 - a set of practices used in agile project management that emphasize daily communication and the flexible reassessment of plans that are carried out in short, iterative phases of work

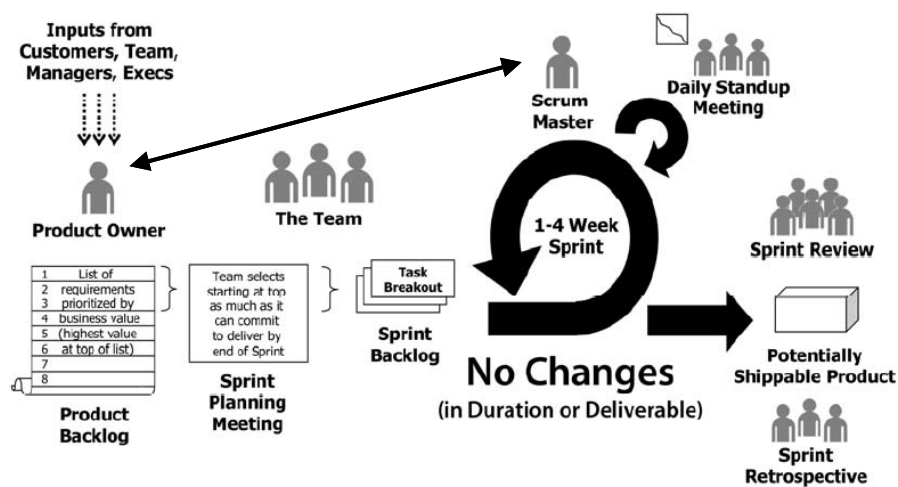


Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

31

Scrum



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

32

Scrum

- Scrum structures product development in cycles of work called **Sprints**,
- Sprints are typically 1-4 weeks in length and the duration is fixed (if completed or not)
- Scrum emphasizes producing working product at the end of the Sprint is really “done”; in the case of software, this means code that is fully tested and potentially shippable.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

33

Scrum

- Requirements are defined as **user stories** and prioritized and documented as **product backlog**.
 - User story:
 - concise, clear descriptions of the functionality in terms of its value to the end user of the product
“As a (role), I want (feature), so that (benefit).”
 - Product backlog (wish list)
 - Collection of user stories
 - Exists (and evolves) over the lifetime of the project
 - Continuously updated by the Product Owner to reflect changes

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

34

Scrum

- Primary Roles
 - Product owner
 - Scrum Master
 - Team

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

35

Scrum

- The process
 - Have product backlog
 - Prioritized list of what's required.
 - It is required to make prioritization and decisions (by Product Owner)
 - Sprint planning
 - Facilitated by the Scrum Master.
 - Product Backlog can vary significantly in size; however, the larger products will often be broken into smaller ones.
 - Team select items from the Product Backlog and commit to complete by the end of the sprint.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

36

Scrum

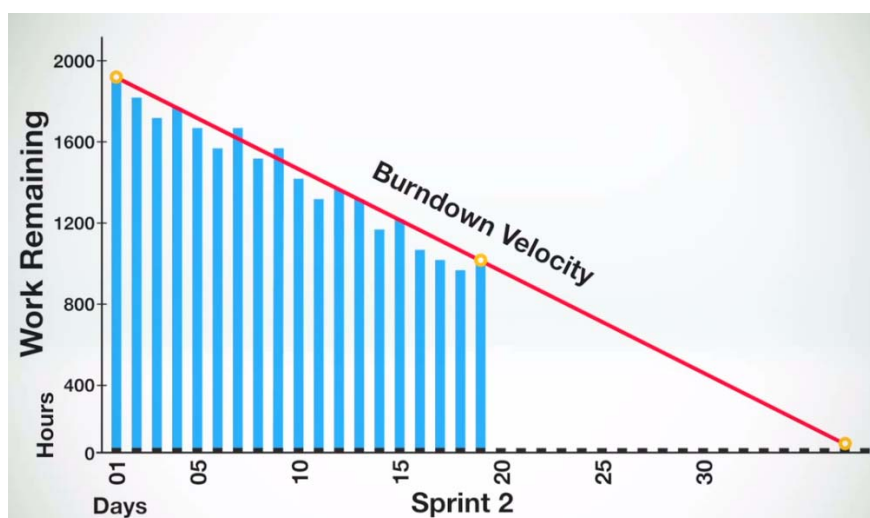
- The process (continued)
 - Daily scrum meeting is:
 - a short (15 minutes or less) stand-up meeting that happens every work day,
 - an opportunity for team to report to itself on progress and obstacles,
 - One by one, each member of the team reports three (and only three) things to the other members of the team
 - No discussion but only reporting. The Scrum Master take notes and makes sure solutions are delivered
 - Sprint Burndown Chart:
 - is used to track the progress of a sprint,
 - shows the amount of work that has been completed in an epic or sprint, and the total work remaining,
 - are used to predict your team's likelihood of completing their work in the time available,

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

37

Scrum



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

38

Scrum

- The process (continued)
 - Sprint review
 - the team demos what they've built during the Sprint (not presentation)
 - Sprint Retrospective is
 - meeting that occurs at the end of each sprint,
 - review of the process (the sprint) is made by discussing on discuss what went well during the previous sprint cycle and what can be improved for the next sprint,
 - Starting the Next Sprint
 - Following the Sprint Review Meeting, the Product Owner takes all the input, as well as all new priorities that have appeared during the Sprint, and incorporates them into the Product Backlog; new items are added, and existing ones are modified, reordered, or deleted. Once this updating of the Product Backlog is complete, the cycle is ready to begin all over again, with the next Sprint Planning Meeting.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

39

Scrum

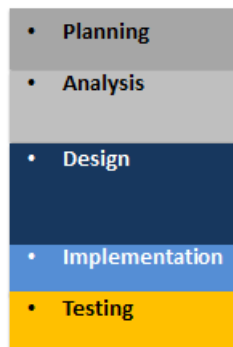
- The process (continued)
 - Release Planning
 - Sprints continue until the Product Owner decides the product is almost ready for release

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

40

From Agile to DevOps



Development



Operations

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

41

From Agile to DevOps



- In earlier process models, requirements were expected to be captured initially in its complete form from the client/users
 - Later it was noted that requirements will change along the process and the notion of stakeholder include many participants in addition to those who owns the system (the client) or the end-users
- Agile, gives more value to meeting requirements than contract negotiation – hence changes are welcome
 - As the results it is common to constantly adjust requirements
 - changes come not only from the business and stakeholders, but also based on the feedback collected from the development team
 - The development team basically starts reflecting beyond technical aspect as they learn more about the system they are developing
- Agile Phases are **plan, code, test** and **review**

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

42

What is DevOps?

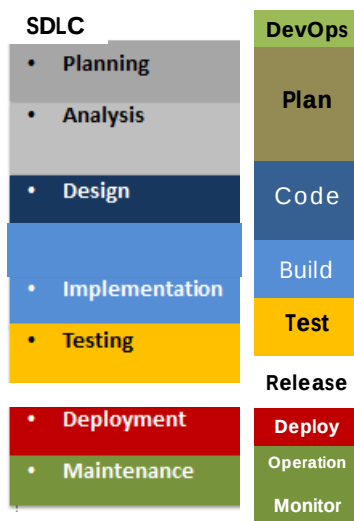
- DevOps is a Philosophy and Practice
 - focused on agility
 - So it focuses on collaboration between the client and development team
 - It also focuses on collaboration between the development team and the operation team
 - Practice automation within IT and development team processes
 - Each phase involves tools

Dagmawi L.
(courses.dag@gmail.com)

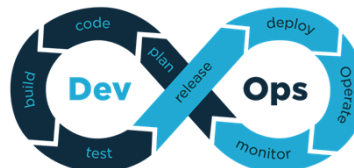
Software Engineering

43

Phases in DevOps



- In DevOps, requirements are not only shaped by the client and stakeholders and the development team, but also by feedbacks coming from operations and monitoring stage.
 - This means the initial requirements affect the development, and the development will affect the operations (how things used to be done), and the operations will give feedback affecting the requirements.



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

44

Phases in DevOps

- Plan
 - At this stage the client and the development team defines the requirements
 - Features of the system to-be is listed
 - User stories (wishlist) is created and recorded
 - Product backlog is created
- Code
 - Design will be conducted and codes for the designed units are created
- Build
 - Units are integrated and are built as one package
- Test
 - Testing will be conducted (end-to-end) test will be done and units are tested with black box strategy

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

45

Phases in DevOps

- Release
 - The version of the build and new features will be recorded and confirmation is given to deploy the build
- Deploy
 - The software is packaged after release and deployed to the production server
 - i.e., the software is transferred from the developers' machine to the client's platform
- Operate
 - Operation Team does the required configuration at the respective platform (server) and facilitates the platform (e.g., network)
- Monitor
 - Issues will be recorded and assessed for the possible impact on the end-user

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

46

Modelling Tools for OOSE

- Modelling is the process of creating **model** that represents things in the real world or concepts in a certain domain with the intention of allowing ideas to be investigated.
 - Model is used to show the abstraction of what is being modelled.
 - Abstraction provides us with a specific (partial but relevant **perspective**) of the thing or concept being modelled.
- UML – Unified Modelling Language is used to document and communicate models being created during software development process.
- Note that the software can be sought from various perspective (e.g., data, process, modules, platform, etc.)
 - As it is difficult to represent all these perspectives in one model, UML provides various diagrams for each perspectives.
 - As a language, it has its own rules (syntax)

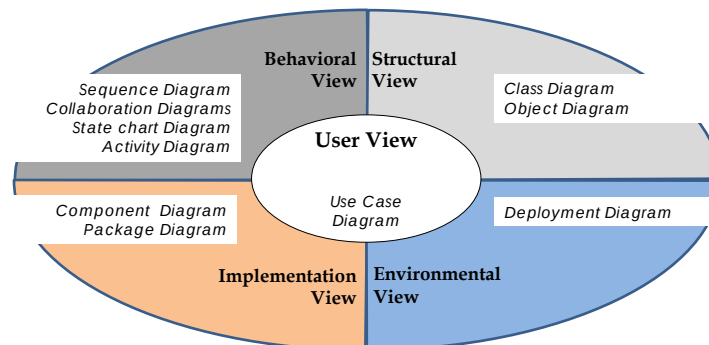
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

47

Modelling Tools for OOSE

- Modelling perspectives in OOSE and diagrams provided in UML



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

48

2. Requirements

Software Engineering

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

49

Requirements Elicitation

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

50

Requirements Elicitation

- What is Requirement?
 - IEEE definition (610.12)
 - “1. a condition or capability needed by a user to solve a problem or achieve an objective.”
 - What the user needs to satisfy the responsibility of the role in the system
 - “2. a condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed document.”
 - What the environment needs
 - What the task demands

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

51

Requirements Elicitation

- NOTE: it is not about what the customer or user wants!
 - It is not personal need rather it is system-wide need
- Requirements are
 - Specification
 - of what should be implemented
 - defining how the system should behave
 - Constraints on the development of the software

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

52

Requirement Type

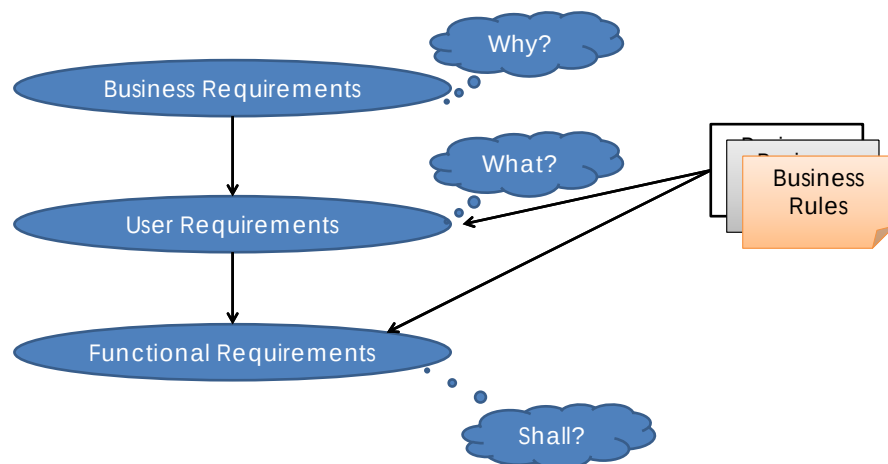
- Business Requirements
 - Why?
 - *Documented as vision and scope document*
- User Requirements
 - What?
 - What would the user be able to do with this system
 - use-cases are useful to describe requirements
 - General description of functional requirements
 - *Documented as use-case*
- Functional Requirements
 - What to be implemented
 - Can be derived using the use-case
 - *Documented as Software Requirement Specification (SRS)*

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

53

Requirement Type



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

54

Requirement Type

- Primarily - the business requirement defines why the software is needed, but also defines
 - The features needed
 - The business rules to be respected
 - E.g. Library system:
 - The business requirement defines why the software (online system) is required, what would be the scope of the system, etc.
 - members can reserve books online (The business requirement → improved satisfaction)

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

55

Requirement Type

- Business Rules
 - are principles, policies, guidelines, standards, etc. to be respected when performing a task or to be part of the system
 - Defines
 - Rules to be respected as functional requirements are executed
 - Rules to be respected to meet the business requirements and the quality attributes
 - Examples
 - in a university-library,
 - a *member* should be active student or staff member of the university
 - » *This requirement shows the business rules to be satisfied in order to be member of the library (active "student" or "staff members")*
 - a *member* should return the book he/she borrowed within 2weeks (14days)
 - *This requirement shows the business rules to be respected after a book is borrowed*
 - Can be associated with various types of requirements also used to identify and clarify requirements

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

56

Requirement Type

- Business Rules (continued)
 - functional
 - Examples
 - the system shall record the **book-barcode**, *return-date* and the *member-ID* in the borrowed list when a member borrow book from the library
 - the system shall record the **ISBN**, *reservation-date*, *reservation-time* and the *member-ID* in the reserved list when a member reserve a book in the library
 - Nonfunctional
 - Examples
 - the system shall allow a member to borrow **a maximum of four books**
 - the system shall allow a member to reserve **a maximum of one book**
 - the record made during the reservation of book shall be turned into borrowed list **in 48hours** otherwise the record shall be deleted
 - pseudo-requirements
 - Examples
 - the online system shall be built using php and MySQL database

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

57

Requirement Type

- Requirement can be generally classified as functional or nonfunctional
 - Functional focus on what task to be done
 - Nonfunctional explains the **customizable** business rules (e.g. password length, response time, etc.) to be maintained while doing the task and the required environment
 - Business rules
 - System requirements which further describes features of the product

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

58

Requirement Type

- NFR
 - Business rules
 - Corporate policies
 - E.g.
 - » how frequently the user shall change the password
 - » Password length
 - Industry standards
 - Government policies
 - Quality attributes (the “ilities”)
 - Usability
 - Maintainability
 - Dependability
 - Portability
 - As well as performance, cost etc.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

59

Good Characteristics of Requirement

- How do we know we have good requirement?
 - Verification
 - Are we building the product right?
 - refers to the set of activities that ensure that software (the product) correctly implements a specific function
 - What we implement, is it correct?
 - Validation
 - Are we building the right product?
 - refers to a different set of activities that ensure that the software that has been built is traceable to customer requirements
 - Did we implement what the user requires?

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

60

Required Characteristics

- Correctness
 - A specification is correct if it represents the client's view of the system.
 - Each requirement shall describe something actually needed by the customer; and that is related to the business requirement.
 - The model describes the reality of interest to the client, not another reality.

Example Business Requirement:

- To improve customer satisfaction by providing features like remote book reservation and searching.

Example User Story:

- As a Librarian I want to record books borrowed by members in the borrow list so that I can trace who borrowed which book

The example user story is true and it may support the user's task (Librarian's), but is it valid in view of the business requirement?

Perhaps it needs to be corrected as:

- As a Librarian I want to record books borrowed by members in the borrow list so that I can easily have information on the book inventory and the availability of a book

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

61

Required Characteristics

- Consistency
 - Should not contradict itself
 - All concepts in the model correspond to phenomena of the same reality
- Completeness
 - Every phenomenon of interest is described in the model by a concept
 - Externally - all desired properties are present.
 - Internally - no undefined references
- Clarity
 - Properties and items in the system shall be clearly described
 - All concepts in the model correspond to exactly one phenomenon
- Requirements shall be verifiable (testable)
- Requirements shall be traceable
- Feasible and Realistic
 - The model describes a reality that can exist
 - The requirement shall be implementable in the context of the environment

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

62

Required Characteristics

- Example
 - What is wrong with this requirement?
 - What is the Cataloguer?
 - Person, Hardware or Software ...
 - “required information of the book” ?
 - Type and detail of information shall be described

Identified Requirement
in the Catalogue Section
Role Identified: Cataloguer

Role Description: The Cataloguer
records required information of
the book.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

63

Required Characteristics

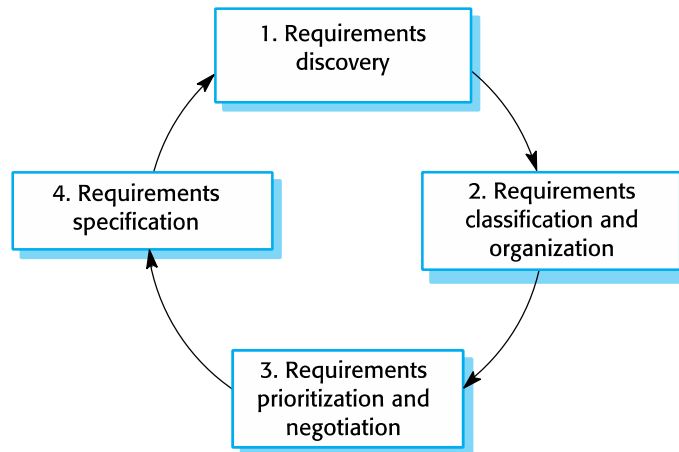
- Example (continued)
 - Identified Requirement in the Catalogue Section
 - Role Identified: Cataloguer
 - Process Identified: Cataloging
 - Role Description: The Cataloguer is a person who is responsible to record new book information in the database and produce catalogue card. The required book information comprise ISBN, Book Title, Name of Author, Publisher, Barcode and Price of the book.
 - The ISBN is unique code assigned to a book, which consists Latin character and digits.
 - The Barcode is produced after the catalogue card is produced
 - The Catalogue card is produced in three forms ...

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

64

Elicitation and Analysis Process



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

65

Elicitation and Analysis Process

- Requirements Elicitation:
 - Definition of the system in terms understood by the customer (*“Problem Description”*)
- Requirements Analysis:
 - Technical specification of the system in terms understood by the developer (*“Problem Specification”*)

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

66

Tasks in Elicitation

- Identifying actors/roles
- Identifying scenarios
- Identifying use-cases
- Refining use-cases
- Identifying relationships among use-cases
- Identifying participating objects
- Identifying nonfunctional requirements
- *The tasks and the order may vary based on the method*
 - *Are we following object Oriented approach or ...?*
 - *User centered or task centered or data centered*
 - *Note that this determines what to identify first (i.e., the class or the actor or the use-case or ...)*
 - *Not that one factor to select among the various method is the type of system you are developing (i.e., DSS or TPS or MIS or ...)*
 - *Scenario based or not ...?*

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

67

Requirement Identification

- For example [Brugge] present a method that
 - focus on scenario-based requirements elicitation
 - elicit requirements by observing and interviewing users, so that need to identify the users first
 - then represent the user's current work processes as as-is scenarios and then develop visionary scenarios describing the functionality provided by the future system

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

68

Requirement Identification

- Requirement Discovery / Determination / Identification
 - Plan how to do it
 - Gather Information
 - Review Identified Requirements
 - Prototype
 - Development and evaluation
 - JAD

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

69

Requirement Identification

- Background Analysis
 - Learn about the setting, the existing system and the physical processes related to the revised system
 - Who runs what
 - Who reports to whom
 - What is relationship between x and y?
 - E.g. relationship between catalogue section and circulation desk – in a library system
 - The nature and the type of interaction between processes
 - Etc.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

70

Requirement Identification

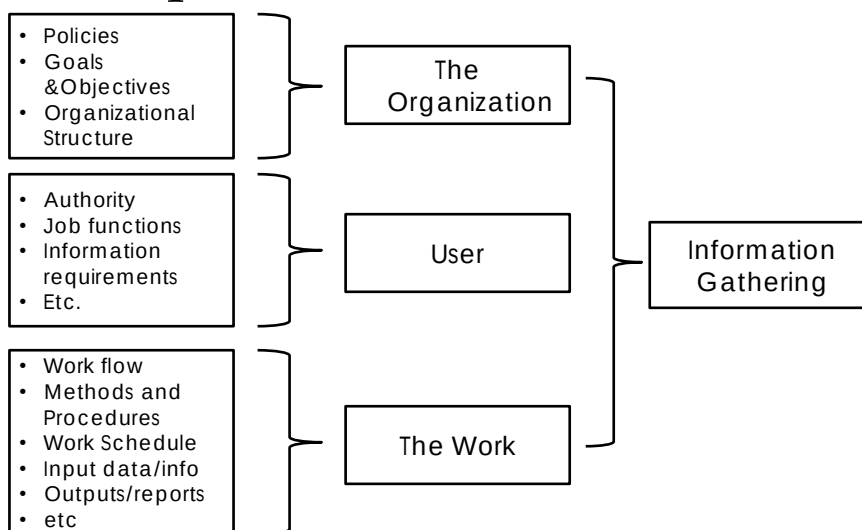
- Information Gathering
 - What kind of information do we need?
 - About the firm
 - About the users
 - About the process
 - Techniques
 - Site Observation and Interview
 - Document Review/Investigation
 - Interview
 - Questionnaire

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

71

Requirement Identification



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

72

Requirement Identification

- Traditional Methods for Requirement Identification
 - Interview
 - Questionnaire
 - Site/User observation
 - Document Review

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

73

Requirement Identification

1. Interview
 - Guideline
 - Plan the Interview
 - Identify interviewee: appointment, lead questions
 - Think about what you need to find out
 - Listen carefully and take notes (tape record if permitted)
 - Review notes within 48hrs of the interview
 - Be neutral
 - Seek diverse views
 - Interview Questions
 - Open-ended questions
 - Close-ended questions
 - **Thank the Interviewee** at the beginning and at the end

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

74

Requirement Identification

2. Questionnaires

– Choosing respondents

- if more people to survey decide which set of people to send questionnaire to or which questionnaire to send to which group of people
 - Convenient: people at a local site or willing to get surveyed
 - Random sample: select any person from a list
 - Purposeful sample: select people who satisfy certain criteria
 - Stratified sample: select random set from each of many categories

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

75

Requirement Identification

	Questionnaires	Interview
Useful for ...	Specific purpose	General information gathering
Type of questions	Close-ended	Open-ended as well as Close-ended
Analysis	Easy	Difficult
Clarity	Has to be extremely clear	Can be clarified on-site
Time	Required less time	Requires more time
Participant	Could involve many respondent	Could only involve only representatives of the target group

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

76

Requirement Identification

3. Site/User Observation

- People cannot always be trusted to reliably report their own actions
 - They may think some details are commonsense since they have done it for a very long time
 - They are afraid of the consequence of their answers
- Often difficult to obtain unbiased data
- People often work differently when being observed
 - Telling the task manual but doing something else
- Formal systems Vs Informal system

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

77

Requirement Identification

4. Document Review

- Four types of useful documents
 - Written work procedure for an individual or a work group
 - Describes how a job is performed
 - Includes data and information used and created in the process of performing the job or task
 - Business form
 - Explicitly indicate what data flow in or out of a system and which are necessary for the system to work
 - Report generated by current systems
 - Enables the analyst to work backwards from the report to the data that generated it – company's performance is past years
 - Description of current information system
 - If the current system is computer based

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

78

Requirement Identification

- Modern Methods for Requirement Identification
 - Joint Application Design (JAD)
 - Prototype
 - CASE tools

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

79

Requirement Identification

1. JAD

- Similar to group interview as it brings together key users, managers and systems analysts
- Purpose: collect system requirements simultaneously from key people
- Particular structure of roles and agenda is followed and analysts control the sequence of questions answered by users
- Conducted off-site to keep participants away from distractions
- may last from four hours to an entire week and may consist of many weeks

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

80

Requirement Identification

1. JAD (continued)

– Participants

- Session Leader – organizes and runs the JAD
- Users – key users of the current system
- Managers of the workgroups who use the current system
- Sponsor – needed to cover expenses
- Systems Analysts – to learn from users and managers
- Scribe – takes notes
- IT Staff – other IT staff like programmers, database analysts

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

81

Requirement Identification

2. Prototyping

- Repetitive process
- Rudimentary version of system is built
- Replaces or augments SDLC
- Goal: to develop concrete specifications for ultimate system

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

82

Requirement Identification

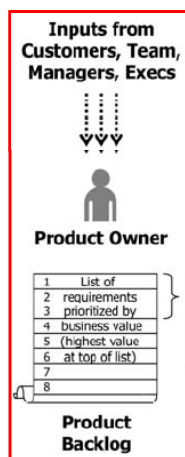
- In the case of waterfall models, requirement identification focus on “what” is needed by the “system”, or “what” the “system” shall fulfill.
 - hence scenarios can be used to identify, primarily, the system's requirements
- In the case of agile methods (scrum), instead of the system's requirements, the focus is on the users' requirements.
 - Hence user stories are used to identify, primarily, “what” and “why” the user demands a certain functionality.
 - A user story is the smallest unit of work with an end goal,
 - a functionality expected by end user or the customer.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

83

Requirement Identification



- The purpose of a user story is to articulate how a piece of work will deliver a particular value back to the customer (stakeholders).
- In scrum, the **Product Owner** is responsible to collect the user stories and prioritize them as a Product Backlog (whish list)

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

84

Requirement Identification

- If we are following scrum, user stories are the requirements which should be identified from roles (actors) in the system.
 - However, actor is not necessarily human user but it can also be:
 - External system
 - Device component
- User stories shall be related and contextualized to a particular scenario (use case)

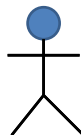
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

85

Actors

- The **actor** in your system shall be considered as roles outside the boundary of the system who are initiating a certain action (use-case) or communicated by the use-case
- Actor is denoted by a sticky person



Actor ID Goes Here

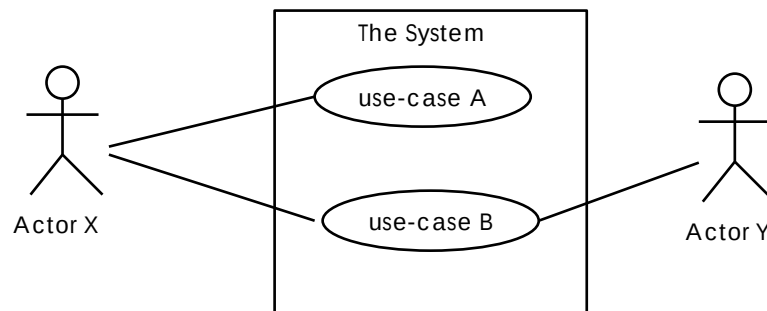
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

86

Actors

- Note that use-cases are within the boundary of the system while actors are outside the boundary
- An actor can also be involved in more than one use-cases



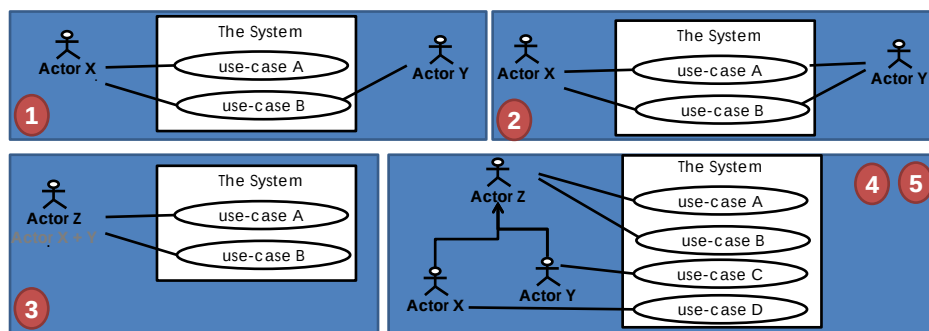
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

87

Actors

- Identifying actors is an iterative process, just like many of the SE tasks
- For Example,
 1. Let's assume that you have identified more than one actor (Actor X and Y)
 2. Eventually you may noted that all the actors are participating in the same way
 3. Therefore you need to represent those actors using common name (Actor Z) = Actor X + Y
 4. In the process you may identify a new use-case in which only one of the merged actor is participating
 5. Therefore you need to have specialization of the actor
 6. And so on...



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

88

Actors

- NOTE: most of the time actors are aligned to a human users, but actor can also be
 - an external system, which you strategically decided to be external as you define your use-case
 - a device, which perform a certain action but often a reason to initiate another use-case

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

89

Scenario

- “A narrative description of what people do and experience as they try to make use of computer systems and applications” [M. Carrol, Scenario-based Design, Wiley, 1995]
- Scenario can be
 - As-is
 - Visionary
 - Test / evaluation
 - training
- A concrete, focused, informal description of a single feature of the system used by a single actor.
- Scenarios can have many different uses during the software lifecycle
 - *Requirements Elicitation*: As-is scenario, visionary scenario
 - *Client Acceptance Test*: Evaluation scenario
 - *System Deployment*: Training scenario.

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

90

Example

- Scenario

- reserve_book (Main scenario)
 - expired_membership_reserve_book(Exception)
 - print_reservation_code (Extension)
- borrow_book (Main scenario)
 - borrow_for_spot_reading (Extension)
 - borrow_for_takeaway (Extension)

Possibly appears as
alternative flow
of use-case

Possibly appears as
generalization of use-
case

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

91

Heuristics: How to find use-cases

- Select a narrow vertical slice of the system (i.e. one scenario)
 - Discuss it in detail with the user to understand the user's preferred style of interaction
- Select a horizontal slice (i.e. many scenarios) to define the scope of the system.
 - Discuss the scope with the user
- Use illustrative prototypes (mock-ups) as visual support
- Find out what the user does
 - Task observation (Good)
 - Questionnaires (Bad)

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

92

Use-case

- If scenario based approach is followed, then the use-case is aggregation of scenario
- But a single scenario also can form the use-case



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

93

Use-case

- Use-case is defined as the internal part of the system having a specific goal to achieve
 - A use-case does specific task
- Note that a certain actor can be involved into more than one use-cases
 - Whether the use-cases are sequentially dependent or not, it is recommended to associate a use-case with specific goal

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

94

Use-case

- We have seen that a use-case is represented in UML using oval shape having the name inside the shape
- Several use cases collectively build the system can be depicted with a use-case diagram
- Yet each use-case need to be described
 - No specific rule on how the use-case description should be organized
 - It can be narrated in a paragraph or two
 - It can be presented in structured form

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

95

Use-case

- The use-case description need to include and specify
 - Use-case ID,
 - which facilitate traceability to requirement/s
 - Use-case name
 - Goal
 - Participating actor/s
 - Actor that initiate the use-case
 - Actor that might be communicated by the use-case
 - Flow of event
 - Pre-condition (i.e., entry condition)
 - What should be fulfilled in order to start the use-case
 - Exception (alternative flow)
 - Post-condition (exit condition),
 - when do you say the use-case is done?
 - What should be fulfilled when the use-case is completed

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

96

Use-case Refinement and R/P

- use-case refinement
 - Validation and verification shall be done together with the user – actually by the user/owner of the system
 - Validate
 - Are we building the right system?
 - Have we identified the right use-case?
 - Have we included what the user needs?
 - Verify
 - Are we building the system right?
 - Have we used the correct terminologies?
 - Have we followed the correct procedure (flow of event)?
- Use-case Relationship
 - Include / use
 - Extend
 - Generalization/ Specialization

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

97

Use-case Refinement and R/P

- Use-case Relationship
 - It is worth to identify the possible relationship or dependencies between use-cases as part of the refinement process
 - For instance, we can avoid redundant descriptions which are appearing in the several use-cases we identified in the process
 - Avoiding redundancy helps to achieve consistency

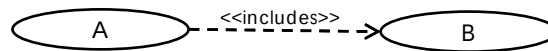
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

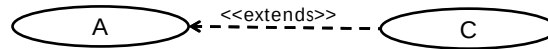
98

Use-case Refinement and R/P

- Include / use
 - Represented by using the include or use stereotype
`<<includes>>` or `<<uses>>`
 - This type of dependency or relationship is used to indicate that the main use-case (A) **needs** the flow of events described in the included use case (B) in order to perform and achieve its goal **or else** those flow of events should be listed in the main use case



- Extend
 - Represented by using the include or use stereotype
`<<extend>>`
 - This type of dependency or relationship is used to indicate that the main use-case (A) may benefit from the flow of events described in the extended use-case (C) while executing to achieve its goal



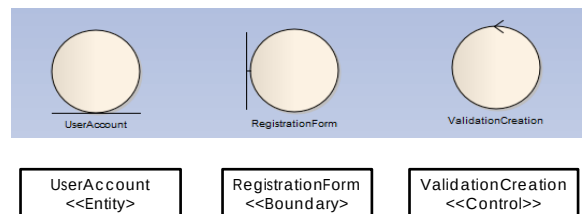
Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

99

Object/Class

- Identifying objects from use-case
 - The three type of objects



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

100

Object/Class

- Boundary Class
 - Represents the interfaces between the actors and the system
 - used to model interaction between the system's surroundings and its inner workings
 - Common boundary classes include windows, communication protocols, printer interfaces, sensors, and terminals

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

101

Object/Class

- Entity Class
 - Manages the information the system need to provide the required functionality
 - used to model information and associated behavior that must be stored
 - **not specific to one** use-case realization

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

102

Object/Class

- Control Class
 - Represents the use-case logic and coordinates the other classes
 - Separates the interface classes from the business logic classes
 - Useful to decouple the dependency between boundary and entity class
 - provide coordinating behavior in the system

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

103

Example (Business Requirement)

- The software under development is library web application
- Business Requirement
 - Objective:
 - Improving accessibility of the library service
 - Automating the circulation desk and the cataloging section of the library
 - Improving customer satisfaction by allowing members to reserve book they want to borrow
 - Scope:
 - Library services that are desired to be accessible includes searching the library catalogue for anyone and reservation service for members
 - The automation is limited to support
 - Librarian while lending books and managing the Library Members
 - Cataloguer while adding book to the catalogue
 - members while reserving book to be borrowed
 - public users while searching the catalogue

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

104

Example (User Requirement)

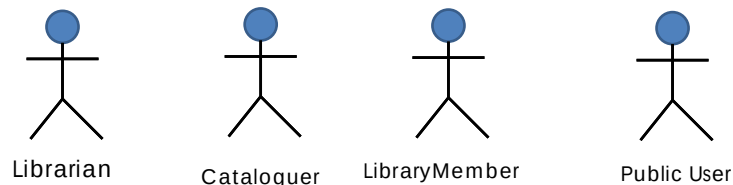
- Librarian needs the software while
 - Lending book
 - Managing member, i.e.
 - Adding member
 - Clearing member (issue clearance)
 - Disabling membership
- Cataloguer needs the software for
 - Adding (Cataloging) book to the Library Catalogue
- Member needs the software for
 - Searching books in the library
 - Reserving books
- Public User needs the software for
 - Searching books in the library

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

105

Example (Identifying Actors)



Also, these actors should be well described

Note:

- we focus on the role of the actor
- the actor could be even a system or hardware

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

106

Example (identifying use-case)

use-case name	ReserveBook
Participating actor	- Initiated by: LibraryMember - Communicated with: None
Entry condition	1. The LibraryMember has signed-in
Flow of events	1. NIBAB presents the home page with menu 2. The LibraryMember selects the "My Reservations" menu under the "Online Services" tab 3. NIBAB opens the "Reservations Page" 4. The LibraryMember clicks on "Reserve New Book" 5. NIBAB presents "reservation form" 6. The LibraryMember completed the reservation form and submitted the form, 7. NIBAB adds the selected book to the reservation log file
Exit Condition	8. The LibraryMember receives acknowledgement and "Service Ticket Number"
Special requirements	• NIBAB give option to print the Service Ticket Number or Send to the LibraryMember's email • Maximum of two books can be reserved

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

107

Example (use-case refinement)

use-case name	ReserveBook
Participating actor	- Initiated by: LibraryMember - Communicated with: None
Entry condition	1. The LibraryMember has signed-in
Flow of events	1. NIBAB presents the home page with menu <i>that consists "Online Services", "Membership Management" and "Member Support"</i> 2. The LibraryMember selects the "My Reservations" menu under the "Online Services" tab menu 3. NIBAB opens the "Reservations page" requesting the user for ISBN of the book to be added to the "reservation list" 4. NIBAB presents "reservation form" 5. The LibraryMember entered the ISBN of the book and clicks the "reserve book" button completed the reservation form and submitted the form, 6. NIBAB adds the selected book to the reservation log file
Exit Condition	8. The LibraryMember receives acknowledgement and "Service Ticket Number" 9. The title and author of the reserved book appeared at the end of the "reservation list"
Special requirements	• NIBAB give option to print the Service Ticket Number or Send to the LibraryMember's email • Maximum of two books can be reserved

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

108

Example (use-case relationship)

use-case name	ReserveBook
Participating actor	- Initiated by: LibraryMember - Communicated with: None
Entry condition	1. The LibraryMember has signed-in
Flow of events	1. NIBAB presents the home page with menu <i>that consists "Online Services", "Membership Management" and "Member Support"</i> 2. The LibraryMember selects the "My Reservations" menu under the "Online Services" menu 3. NIBAB opens the "Reservations page" requesting the user for ISBN of the book to be added to the "reservation list" 4. The LibraryMember entered the ISBN of the book clicks on "Reserve New Book" button, <i>extend SearchBook, include CheckAvailability</i> 5. NIBAB adds the reserved book to the reservation log file, <i>include ValidateReservaion</i>
Exit Condition	6. The LibraryMemer receives acknowledgement and "Service Ticket Number" 7. The title and <i>author</i> of the reserved book appeared at the end of the "reservation list"
Special requirements	• NIBAB give option to print the Service Ticket Number or Send to the LibraryMember's email • Maximum of two books can be reserved
Alternative Flow	5a. NIBAB reports that the LibraryMember has already reserved two books, <i>extend ReleaseReservedBook</i>

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

109

Example (identifying classes)

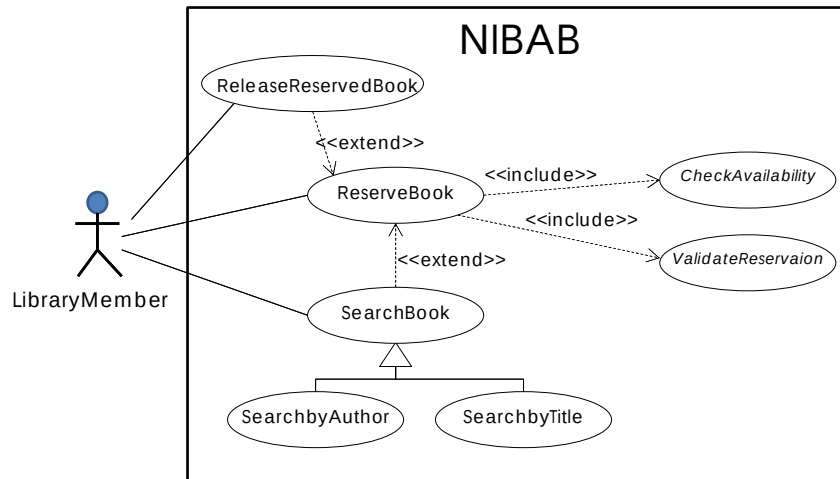
use-case name	ReserveBook
Participating actor	- Initiated by: LibraryMember - Communicated with: None
Entry condition	1. The LibraryMember has signed-in
Flow of events	1. NIBAB presents the "Home Page" with menu <i>that consists "Online Services", "Membership Management" and "Member Support"</i> 2. The LibraryMember selects the "My Reservations" menu under the "Online Services" menu 3. NIBAB opens the "Reservations Page" requesting the user for ISBN of the book to be added to the "reservation list" 4. The LibraryMember entered the ISBN of the book and clicks on "Reserve New Book" button, <i>extend SearchBook, , include CheckAvailability</i> 5. NIBAB adds the reserved book to the reservation log file, <i>include ValidateReservaion</i>
Exit Condition	6. The LibraryMemer receives acknowledgement and "Service Ticket Number" 7. The title and <i>author</i> of the reserved book appeared at the end of the "reservation list"
Special requirements	• NIBAB give option to print the Service Ticket Number or Send to the LibraryMember's email • Maximum of two books can be reserved
Alternative Flow	6a. NIBAB reports that the LibraryMember has already reserved two books, <i>extend ReleaseReservedBook</i>

Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

110

Example (system from specific actor view)



Dagmawi L.
(courses.dag@gmail.com)

Software Engineering

111